
SweetAlert2 con Razor Pages

Confirmaciones, avisos, toasts y operaciones asíncronas accesibles con JavaScript mínimo

Finalidad

Manual orientado a integrar SweetAlert2 en proyectos ASP.NET Core Razor Pages sin romper formularios, antifalsificación, validación ni accesibilidad.

Al terminar este manual podrás:

- Distinguir SweetAlert clásico de SweetAlert2 y cargar la biblioteca correctamente.
- Comprender Swal.fire, Promises y el resultado de confirmar, cancelar o negar.
- Crear una única confirmación reutilizable para todos los formularios de borrado.
- Mostrar mensajes de TempData tras el patrón POST-Redirect-GET.
- Usar input, preConfirm, fetch, loaders, mixins y toasts.
- Integrar botones Bootstrap y modos claro/oscuro.
- Evitar XSS, dobles envíos, bucles de submit y diálogos inaccesibles.
- Probar el comportamiento con Playwright.

Edición didáctica 2026 · SweetAlert2 11 · .NET 10

Índice

1. SweetAlert2, no SweetAlert antiguo	5
2. Instalación	6
CDN	6
LibMan	6
npm	6
Comprobar la carga	6
3. Primera alerta y anatomía de opciones	8
text frente a html	8
4. Promises, await y resultado	9
Motivos de cierre	9
5. Confirmar el borrado en Razor Pages	10
Formulario seguro	10
Listener delegado y reutilizable	10
Por qué requestSubmit	11
6. Handler de borrado y reglas del servidor	12
7. Mensajes TempData después de redirigir	13
Preparar en la vista	13
Mostrar como toast	13
Alert visible o toast	14
8. Mixins y configuración común	15
Pausa del toast	15
9. Integración visual con Bootstrap	16
Modo oscuro	16

10. Inputs y validación del diálogo	17
Tipos	17
11. preConfirm y operaciones asíncronas	18
Antifalsificación con fetch	18
12. Tres decisiones: confirmar, negar y cancelar	20
13. Accesibilidad y experiencia de usuario	21
HTML semántico dentro del popup	21
14. Seguridad	22
XSS	22
Reglas	22
Content Security Policy	22
15. Arquitectura del JavaScript	23
Un módulo sencillo	23
Cuándo usar atributos data	23
Progressive enhancement	23
16. Pruebas con Playwright	24
Cancelar	24
Confirmar	24
Qué verificar	24
17. Errores frecuentes	25
Diagnóstico	25
18. Recetario	26
Información	26
Error recuperable	26
Confirmar salida	26
Copia al portapapeles	26

19. Laboratorio y checklist	28
Actividades propuestas	28
Checklist antes de entregar	28
20. Referencias	29

1. SweetAlert2, no SweetAlert antiguo

SweetAlert2 es una biblioteca moderna de diálogos responsivos y accesibles sin dependencias. Su API principal es **Swal.fire** y devuelve una Promise. Muchos tutoriales antiguos usan **swal(...)**, **type**, callbacks o **result.value**; no deben mezclarse con los ejemplos actuales.

Antiguo o incorrecto	SweetAlert2 actual
swal({...})	Swal.fire({...})
type: 'warning'	icon: 'warning'
result.value	result.isConfirmed / isDenied / isDismissed
callback	await o .then(result => ...)
closeOnConfirm	resolver Promise y decidir la acción

Nombre del manual

En conversación suele decirse SweetAlert, pero los proyectos aquí tratados utilizan SweetAlert2. Comprueba siempre el paquete y la documentación antes de copiar opciones.

2. Instalación

CDN

HTML

```
1 <link rel="stylesheet"
2     href="https://cdn.jsdelivr.net/npm/sweetalert2@11/dist/sweetalert2.min.css">
3
4 <!-- Antes de los scripts propios que usan Swal. -->
5 <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
6 <script src="~/js/sweetalert.js"></script>
```

LibMan

JSON

```
1 {
2   "version": "1.0",
3   "defaultProvider": "cdnjs",
4   "libraries": [
5     {
6       "library": "sweetalert2@11",
7       "destination": "wwwroot/lib/sweetalert2/"
8     }
9   ]
10 }
```

HTML

```
1 <link rel="stylesheet"
2     href="~/lib/sweetalert2/sweetalert2.min.css">
3 <script src="~/lib/sweetalert2/sweetalert2.all.min.js"></script>
```

npm

BASH

```
1 npm install/sweetalert2
2
3 import Swal from 'sweetalert2';
4 import 'sweetalert2/dist/sweetalert2.min.css';
```

En un Razor Pages sin bundler, LibMan o CDN son más sencillos. No cargues/sweetalert2.js y/sweetalert2.all.js a la vez. Fija una versión mayor compatible y revisa actualizaciones en lugar de copiar un archivo anónimo.

Comprobar la carga

JAVASCRIPT

```
1 if (typeof Swal === "undefined") {  
2     console.error("SweetAlert2 no se ha cargado.");  
3 }
```

3. Primera alerta y anatomía de opciones

JAVASCRIPT

```
1  await Swal.fire({
2    icon: "success",
3    title: "Cambios guardados",
4    text: "La pregunta se ha actualizado correctamente.",
5    confirmButtonText: "Aceptar"
6  });
```

Opción	Función
title	título principal
text	texto seguro tratado como texto
html	contenido HTML; exige controlar/sanitizar datos
icon	success, error, warning, info o question
showCancelButton	muestra cancelación
showDenyButton	añade tercera decisión
timer	cierre automático en milisegundos
allowOutsideClick	permite cerrar al pulsar fuera
customClass	clases por elemento

text frente a html

Usa **text** para datos que pueden proceder de usuario, API o base de datos. **html** interpreta etiquetas y puede introducir XSS si se construye mediante interpolación. Si solo necesitas una línea en negrita, pregúntate si el riesgo y la complejidad merecen la pena.

4. Promises, await y resultado

JAVASCRIPT

```
1  const resultado = await Swal.fire({
2      icon: "question",
3      title: "¿Guardar los cambios?",
4      showCancelButton: true,
5      confirmButtonText: "Guardar",
6      cancelButtonText: "Seguir editando"
7  });
8
9  if (resultado.isConfirmed) {
10     guardar();
11 } else if (resultado.isDismissed) {
12     console.info("La persona canceló el diálogo.");
13 }
```

Motivos de cierre

- `isConfirmed`: pulsó confirmar.
- `isDenied`: pulsó negar cuando `showDenyButton` está activo.
- `isDismissed`: se cerró por cancelar, Escape, fuera, botón de cierre o timer.
- `dismiss` contiene el motivo concreto cuando importa distinguirlo.
- `value` contiene el valor de un input o lo devuelto por `preConfirm`.

await necesita async

El listener puede declararse `async`. Las excepciones de operaciones posteriores deben capturarse y producir un mensaje útil; no dejes una Promise rechazada sin observar.

5. Confirmar el borrado en Razor Pages

Formulario seguro

HTML

```
1 <form method="post" asp-page-handler="Eliminar"
2     asp-route-id="@categoria.Id"
3     data-confirmar-borrado
4     data-nombre="@categoria.Nombre"
5     class="d-inline">
6 <button type="submit" class="btn btn-sm btn-outline-danger"
7     aria-label="Eliminar @categoria.Nombre">
8     <i class="bi bi-trash" aria-hidden="true"></i>
9 </button>
10 </form>
```

El Tag Helper genera la URL y el token antifalsificación. La operación sigue siendo POST. SweetAlert2 solo confirma la intención; no sustituye la validación, autorización ni reglas del handler.

Listener delegado y reutilizable

wwwroot/js/sweetalert.js

```
1 document.addEventListener("submit", async event => {
2     const formulario = event.target.closest("form[data-confirmar-borrado]");
3
4     if (!formulario || formulario.dataset.confirmado === "true") {
5         return;
6     }
7
8     event.preventDefault();
9
10    const nombre = formulario.dataset.nombre ?? "este registro";
11    const resultado = await Swal.fire({
12        icon: "warning",
13        title: "¿Eliminar el registro?",
14        text: `Se eliminará ${nombre}. Esta acción no se puede deshacer.`,
15        showCancelButton: true,
16        confirmButtonText: "Eliminar",
17        cancelButtonText: "Cancelar",
18        reverseButtons: true,
19        focusCancel: true
20    });
21
22    if (resultado.isConfirmed) {
23        formulario.dataset.confirmado = "true";
24        formulario.requestSubmit(event.submitter);
25    }
26 });
```

Por qué requestSubmit

requestSubmit vuelve a iniciar el envío como si se pulsara el botón: conserva validación, submitter y evento. La marca data-confirmado evita un bucle. El método submit() envía directamente, omite el evento y la validación de restricciones, y puede quedar oculto por un control con name=submit.

Una sola implementación

No copies onclick en cada botón. La delegación sobre document funciona también para formularios añadidos después y concentra textos, accesibilidad y correcciones en un fichero.

6. Handler de borrado y reglas del servidor

CSHARP

```
1 public async Task<IActionResult> OnPostEliminarAsync(int id)
2 {
3     var categoria = await _context.Categorias
4         .Include(c => c.Preguntas)
5         .SingleOrDefaultAsync(c => c.Id == id);
6
7     if (categoria is null)
8     {
9         return NotFound();
10    }
11
12    if (categoria.Preguntas.Count != 0)
13    {
14        TempData["Error"] =
15            "No se puede eliminar una categoría que contiene preguntas.";
16        return RedirectToPage();
17    }
18
19    _context.Categorias.Remove(categoria);
20    await _context.SaveChangesAsync();
21    TempData["Exito"] = "Categoría eliminada.";
22    return RedirectToPage();
23 }
```

- Busca de nuevo el recurso en el servidor; el DOM no es una fuente de verdad.
- Autoriza la operación sobre ese recurso.
- Comprueba relaciones o restricciones antes de Remove.
- Maneja concurrencia: el registro puede haber cambiado desde que se mostró.
- Redirige tras éxito o error recuperable para aplicar POST-Redirect-GET.

7. Mensajes TempData después de redirigir

Preparar en la vista

HTML

```
1 @using System.Text.Json
2 @{
3     string? exito = TempData["Exito"] as string;
4     string? error = TempData["Error"] as string;
5 }
6
7 @if (exito is not null)
8 {
9     <script>
10         window.mensajePagina = {
11             icono: "success",
12             texto: @Html.Raw(JsonSerializer.Serialize(exito))
13         };
14     </script>
15 }
16 else if (error is not null)
17 {
18     <script>
19         window.mensajePagina = {
20             icono: "error",
21             texto: @Html.Raw(JsonSerializer.Serialize(error))
22         };
23     </script>
24 }
```

Mostrar como toast

JAVASCRIPT

```
1 if (window.mensajePagina) {
2     await Swal.fire({
3         toast: true,
4         position: "top-end",
5         icon: window.mensajePagina.icono,
6         title: window.mensajePagina.texto,
7         showConfirmButton: false,
8         timer: 3500,
9         timerProgressBar: true
10    });
11 }
```

JsonSerializer.Serialize produce un literal JavaScript seguro para el texto. No insertes directamente una cadena de TempData entre comillas: una comilla, salto de línea o secuencia especial rompería el script y podría crear XSS.

Alert visible o toast

Mensaje	Patrón
Confirmación no crítica	toast con tiempo razonable
Error que requiere acción	alerta persistente o bloque visible en página
Validación de formulario	mensajes junto a campos; no solo popup
Operación irreversible	diálogo modal de confirmación

8. Mixins y configuración común

JAVASCRIPT

```
1  const Alerta = Swal.mixin({
2    confirmButtonText: "Aceptar",
3    cancelButtonText: "Cancelar",
4    reverseButtons: true,
5    customClass: {
6      confirmButton: "btn btn-primary",
7      cancelButton: "btn btn-outline-secondary me-2"
8    },
9    buttonsStyling: false
10 });
11
12 const Toast = Swal.mixin({
13   toast: true,
14   position: "top-end",
15   showConfirmButton: false,
16   timer: 3500,
17   timerProgressBar: true
18 });
```

Un mixin crea una versión preconfigurada sin modificar opciones globales. Centraliza traducciones, orden de botones y clases Bootstrap. Mantén mixins diferentes para decisiones modales y notificaciones no bloqueantes.

Pausa del toast

JAVASCRIPT

```
1  const toast = await Toast.fire({
2    icon: "success",
3    title: "Cambios guardados",
4    didOpen: elemento => {
5      elemento.addEventListener("mouseenter", Swal.stopTimer);
6      elemento.addEventListener("mouseleave", Swal.resumeTimer);
7    }
8  });
```

Tiempo para leer

Un toast desaparece. No lo uses como único canal para errores complejos ni acciones necesarias. Pausar el timer al pasar el puntero no ayuda a quien usa teclado; conserva información importante en la página.

9. Integración visual con Bootstrap

JAVASCRIPT

```
1  const resultado = await Swal.fire({
2      icon: "warning",
3      title: "¿Eliminar?",
4      showCancelButton: true,
5      confirmButtonText: "Eliminar",
6      cancelButtonText: "Cancelar",
7      buttonsStyling: false,
8      customClass: {
9          actions: "gap-2",
10         confirmButton: "btn btn-danger",
11         cancelButton: "btn btn-outline-secondary"
12     }
13 });
```

`buttonsStyling=false` evita que SweetAlert2 aplique su estilo de botón y deja actuar a Bootstrap. No añadas márgenes contradictorios si `actions` ya usa `gap`. Comprueba foco visible y contraste con cada tema.

Modo oscuro

JAVASCRIPT

```
1  function temaSweetAlert() {
2      return document.documentElement.dataset.bsTheme === "dark"
3          ? "dark"
4          : "light";
5  }
6
7  await Swal.fire({
8      theme: temaSweetAlert(),
9      title: "Información",
10     text: "El tema del diálogo sigue al de la página."
11 });
```

Las versiones recientes admiten `themes`; comprueba que los activos del tema están incluidos según la distribución usada. Como alternativa, define `color/background` o `customClass` mediante variables Bootstrap. No mezcles fondo oscuro con texto fijo oscuro.

10. Inputs y validación del diálogo

JAVASCRIPT

```
1  const { value: nombre } = await Swal.fire({
2    title: "Nombre de la categoría",
3    input: "text",
4    inputLabel: "Nuevo nombre",
5    inputPlaceholder: "Por ejemplo, Ciencia",
6    showCancelButton: true,
7    inputValue: valor => {
8      if (!valor?.trim()) {
9        return "Escribe un nombre.";
10     }
11     if (valor.trim().length > 80) {
12       return "El nombre no puede superar 80 caracteres.";
13     }
14     return undefined;
15   }
16 });
17
18 if (nombre) {
19   console.log(nombre.trim());
20 }
```

Un input dentro del popup es útil para una decisión breve. Para entidades con varios campos, errores de servidor y accesibilidad, un formulario Razor normal es mejor. La validación JavaScript no sustituye la del servidor.

Tipos

- text, email, password, number, tel, url, textarea.
- select y radio con inputOptions.
- checkbox para una confirmación explícita.
- file para una selección puntual, aunque un formulario completo suele ser más claro.

11. preConfirm y operaciones asíncronas

JAVASCRIPT

```
1  const resultado = await Swal.fire({
2    title: "Comprobar código",
3    input: "text",
4    showCancelButton: true,
5    showLoaderOnConfirm: true,
6    allowOutsideClick: () => !Swal.isLoading(),
7    preConfirm: async codigo => {
8      const respuesta = await fetch("/api/codigos/comprobar", {
9        method: "POST",
10       headers: {
11         "Content-Type": "application/json"
12       },
13       body: JSON.stringify({ codigo })
14     });
15
16     if (!respuesta.ok) {
17       Swal.showValidationMessage(
18         `No se pudo comprobar: ${respuesta.status}`);
19       return false;
20     }
21
22     return await respuesta.json();
23   }
24 });
25
26 if (resultado.isConfirmed) {
27   console.log(resultado.value);
28 }
```

preConfirm puede devolver un valor o Promise. Si muestra validationMessage, el diálogo permanece abierto. showLoaderOnConfirm comunica progreso. Deshabilitar el cierre externo durante la carga evita abandonar una operación sin saber qué ocurrió.

Antifalsificación con fetch

JAVASCRIPT

```
1 const token = document.querySelector(  
2   'input[name="__RequestVerificationToken"]')?.value;  
3  
4 await fetch("?handler=Comprobar", {  
5   method: "POST",  
6   headers: {  
7     "RequestVerificationToken": token  
8   }  
9 });
```

Mejor formulario cuando sea posible

Para borrar desde Razor Pages, enviar el formulario existente conserva el token automáticamente y requiere menos código que fetch. Usa fetch cuando realmente necesitas una interacción sin navegación.

12. Tres decisiones: confirmar, negar y cancelar

JAVASCRIPT

```
1  const resultado = await Swal.fire({
2      icon: "question",
3      title: "¿Guardar antes de salir?",
4      showDenyButton: true,
5      showCancelButton: true,
6      confirmButtonText: "Guardar y salir",
7      denyButtonText: "Salir sin guardar",
8      cancelButtonText: "Seguir editando"
9  });
10
11  if (resultado.isConfirmed) {
12      await guardarYSalir();
13  } else if (resultado.isDenied) {
14      salirSinGuardar();
15  }
```

Los botones deben representar opciones mutuamente comprensibles. Cancelar conserva el estado y cierra el diálogo; negar expresa una decisión alternativa. Evita Sí/No cuando el objeto de la respuesta pueda ser ambiguo.

13. Accesibilidad y experiencia de usuario

- Un título breve explica la decisión; el texto aclara consecuencias.
- Los botones usan verbos: Eliminar, Guardar, Cancelar.
- `focusCancel` es prudente para acciones destructivas; no sorprenda al teclado.
- `allowEscapeKey` y `allowOutsideClick` deben elegirse según la criticidad.
- No bloquee la interfaz con una alerta para cada acción trivial.
- No presente información esencial solo durante tres segundos.
- El foco debe volver a un elemento lógico tras cerrar.
- Los colores e iconos complementan el texto, no lo sustituyen.

HTML semántico dentro del popup

SweetAlert2 gestiona `role=dialog`, foco y teclado, pero el contenido personalizado sigue siendo responsabilidad del proyecto. Si insertas una lista, usa `ul/li`; si hay un enlace, su texto debe ser descriptivo; si hay un input personalizado, aporta label.

Fatiga modal

Un diálogo interrumpe. Confirmar todos los clics hace que el usuario aprenda a aceptar sin leer. Reserva confirmaciones para consecuencias relevantes y ofrece deshacer cuando sea viable.

14. Seguridad

XSS

JAVASCRIPT

```
1 // Seguro: el valor se muestra como texto.
2 await Swal.fire({ title: "Usuario", text: nombreRecibido });
3
4 // Peligroso si nombreRecibido no es de confianza.
5 await Swal.fire({ html: `<strong>${nombreRecibido}</strong>` });
```

Reglas

- Usa text para datos dinámicos.
- Si html es imprescindible, crea nodos con textContent o sanitiza con una biblioteca mantenida.
- No muestres stack traces, SQL, tokens ni detalles internos al usuario.
- SweetAlert2 no autoriza ni valida el recurso: el servidor repite todas las comprobaciones.
- La confirmación no protege contra CSRF; conserva antiforgery.
- Evita incorporar scripts desde CDN sin versionado ni política coherente de seguridad.

Content Security Policy

Una CSP estricta puede bloquear scripts inline. Coloca la lógica en archivos .js y pasa datos mediante elementos data-* o JSON seguro. Si usas nonce, debe generarse por respuesta y aplicarse correctamente; no debilites la política con unsafe-inline solo para una alerta.

15. Arquitectura del JavaScript

Un módulo sencillo

JAVASCRIPT

```
1  export function instalarConfirmacionesBorrado() {
2      document.addEventListener("submit", confirmarBorrado);
3  }
4
5  async function confirmarBorrado(event) {
6      const formulario = event.target.closest("[data-confirmar-borrado]");
7      if (!formulario || formulario.dataset.confirmando) return;
8
9      event.preventDefault();
10     const resultado = await mostrarConfirmacion(formulario.dataset.nombre);
11
12     if (resultado.isConfirmed) {
13         formulario.dataset.confirmando = "true";
14         formulario.requestSubmit(event.submitter);
15     }
16 }
17
18 function mostrarConfirmacion(nombre) {
19     return Swal.fire({
20         icon: "warning",
21         title: "¿Eliminar?",
22         text: `Se eliminará ${nombre ?? "el registro"}.`,
23         showCancelButton: true
24     });
25 }
```

Cuándo usar atributos data

data-confirmar-borrado marca comportamiento y data-nombre aporta una cadena. No almacenes objetos enormes ni secretos. Para opciones complejas, usa configuración de módulo. La delegación permite aplicar la misma conducta a preguntas, categorías y otros listados.

Progressive enhancement

Sin JavaScript, el formulario seguiría borrando sin confirmación. Para acciones críticas puedes incluir una página de confirmación de servidor como alternativa. SweetAlert2 mejora la experiencia, pero la aplicación debe seguir siendo segura si el script falla.

16. Pruebas con Playwright

Cancelar

CSHARP

```
1 await fila.GetByRole(AriaRole.Button,
2     new() { Name = "Eliminar" }).ClickAsync();
3
4 Locator dialogo = Page.GetByRole(AriaRole.Dialog);
5 await Expect(dialogo).ToContainTextAsync("no se puede deshacer");
6 await dialogo.GetByRole(AriaRole.Button,
7     new() { Name = "Cancelar" }).ClickAsync();
8
9 await Expect(fila).ToBeVisibleAsync();
```

Confirmar

CSHARP

```
1 await fila.GetByRole(AriaRole.Button,
2     new() { Name = "Eliminar" }).ClickAsync();
3 await Page.GetByRole(AriaRole.Dialog)
4     .GetByRole(AriaRole.Button,
5         new() { Name = "Eliminar" })
6     .ClickAsync();
7
8 await Expect(Page.GetByText("Categoría eliminada"))
9     .ToBeVisibleAsync();
10 await Expect(fila).ToBeHiddenAsync();
```

Qué verificar

- Título, consecuencia y nombres de botones.
- Cancelar no envía POST ni cambia datos.
- Confirmar envía una sola petición.
- El servidor puede rechazar y el mensaje se muestra.
- El foco inicial y el retorno de foco son razonables.
- El diálogo funciona en tema oscuro y viewport móvil.

SweetAlert2 está en el DOM

En Playwright se usa `GetByRole(Dialog)`. El evento `Page.Dialog` es solo para `alert()`, `confirm()` y `prompt()` nativos.

17. Errores frecuentes

Síntoma	Causa probable y corrección
Swal is not defined	script no cargado, ruta errónea u orden incorrecto
Se borra antes de confirmar	no se llamó preventDefault al evento submit
Bucle de confirmación	requestSubmit vuelve a disparar submit; usar marca confirmado
No llega el handler	form action/asp-page-handler incorrecto o botón fuera del form
Falla antiforgery	se reconstruyó fetch sin token; enviar formulario existente
No funciona result.value	ejemplo antiguo; usar isConfirmed/value según contexto
Botones no parecen Bootstrap	buttonsStyling=false y customClass correctos
Popup claro con tema oscuro	configurar theme/color/background o clases
Texto rompe script	serializar JSON; no interpolar TempData directamente
Doble petición	varios listeners u onclick más submit delegado

Diagnóstico

1. Comprueba en Red que CSS y JS devuelven 200.
2. Escribe typeof Swal en consola.
3. Comprueba que el listener observa submit y no solo click.
4. Inspecciona action, method, handler y token del formulario.
5. Mira si existe más de una copia de sweetalert.js o listeners duplicados.
6. Reproduce con Playwright y examina petición y diálogo en la traza.

18. Recetario

Información

JAVASCRIPT

```
1  await Swal.fire({
2      icon: "info",
3      title: "API disponible",
4      text: "Puedes consultar hasta 100 preguntas por petición."
5  });
```

Error recuperable

JAVASCRIPT

```
1  await Swal.fire({
2      icon: "error",
3      title: "No se pudo cargar",
4      text: "Comprueba la conexión y vuelve a intentarlo.",
5      confirmButtonText: "Reintentar",
6      showCancelButton: true,
7      cancelButtonText: "Cerrar"
8  });
```

Confirmar salida

JAVASCRIPT

```
1  const { isConfirmed } = await Swal.fire({
2      icon: "warning",
3      title: "¿Salir de la partida?",
4      text: "Perderás el progreso de la ronda actual.",
5      showCancelButton: true,
6      confirmButtonText: "Salir",
7      cancelButtonText: "Continuar jugando",
8      focusCancel: true
9  });
```

Copia al portapapeles

JAVASCRIPT

```
1  try {  
2      await navigator.clipboard.writeText(urlApi);  
3      await Toast.fire({ icon: "success", title: "URL copiada" });  
4  } catch {  
5      await Alerta.fire({  
6          icon: "error",  
7          title: "No se pudo copiar",  
8          text: "Selecciona la URL y cópiala manualmente."  
9      });  
10 }
```

19. Laboratorio y checklist

Actividades propuestas

1. Instala SweetAlert2 localmente y confirma que funciona sin internet.
2. Marca todos los formularios de borrado con data-confirmar-borrado y usa un solo listener.
3. Añade nombre de registro al texto sin usar html.
4. Muestra TempData Exit/Error mediante un JSON seguro y mixins comunes.
5. Adapta botones a Bootstrap y prueba claro, oscuro y Darkly.
6. Crea una operación con preConfirm que maneje 400 y 500 sin cerrar el popup.
7. Escribe Playwright para cancelar, confirmar y rechazo del servidor.
8. Desactiva JavaScript y comprueba que el servidor continúa protegido.

Checklist antes de entregar

- Solo se carga una versión de SweetAlert2.
- No quedan APIs antiguas: swal, type, result.value para confirmación.
- Los formularios conservan POST, handler y antiforgery.
- El servidor repite validación y autorización.
- No se introduce dato no confiable mediante html.
- Cancelar no cambia estado y confirmar solo envía una vez.
- Mensajes críticos no desaparecen únicamente en un toast.
- Diálogos probados con teclado, móvil y tema oscuro.

20. Referencias

[SweetAlert2: documentación y ejemplos](#)

[Galería de recetas](#)

[Repositorio oficial](#)

[Bootstrap: botones](#)

[Antifalsificación en ASP.NET Core](#)

[Playwright: localizadores](#)